

ULTIMO DOCUMENTO DEL CORSO – ATTO FINALE

PESSIMISTIC COVERAGE

Prefazione

Siamo giunti alla conclusione del corso: questo sarà l'ultimo documento del corso di Ingegneria del Software. Ho cercato di trattare la maggior parte degli argomenti, spiegandoli sempre nel modo più chiaro e semplice possibile. Ora desidero aggiungere alcune considerazioni.

In particolare, la coverage è un argomento da non sottovalutare. Pur essendo estremamente semplice come concetto teorico, negli ambienti industriali la copertura del software rappresenta un parametro di valutazione molto importante. Detto ciò, soffermiamoci su ciò che è necessario sapere per l'esame, cercando di mettere un po' di ordine.

Come comprendere la coverage? Come valutare la copertura del codice?

La coverage è un concetto che trova piena applicazione nella pratica. Il mio consiglio è di installare JUnit e configurare opportunamente il vostro IDE per l'esecuzione dei test.

Nel caso dell'optimistic coverage, è necessario che la copertura logica (nota bene: copertura logica indica la parte applicativa) raggiunga il 100%. In altre parole, la copertura deve essere massima.

Nel PDF precedente, non tutti i codici relativi all'optimistic coverage raggiungevano il 100%; questa scelta è stata intenzionale, per consentire una comprensione graduale del concetto.

All'esame, tuttavia, qualora venga richiesto un caso di optimistic coverage, non è accettabile presentare un codice che non raggiunga la copertura logica totale. Ricordate che nell'optimistic coverage è presente un bug, ma esso non è evidente.

Nell'impossible coverage, invece, il codice logico non può essere interamente coperto. In questo caso, occorre creare un frammento di codice che, per sua natura, non può essere testato completamente.

Un esempio può essere la generazione casuale di numeri con un successivo controllo, oppure l'utilizzo di try-catch in cui si simula una funzione da testare. Tuttavia, in questo caso, non si deve mantenere una copertura logica del 100%.

È fondamentale rispettare la tabella del "Breaking Coverage", poiché rappresenta un riferimento inequivocabile.

Detto questo, procedo ora a spiegare con chiarezza il concetto di pessimistic coverage, in modo che si abbia una comprensione completa di tutte le casistiche richieste all'esame.

Mi auguro che questo documento possa esservi utile.

Nel pessimistic coverage, la copertura deve essere sempre del 100%: non sono presenti bug reali, ma il comportamento del codice può farli sembrare tali.

COSA VUOL DIRE COPERTURA PESSIMISTICA?

Il test **fa pensare** che ci sia un errore logico o funzionale, ma questo **non è effettivamente presente**. Nel **pessimistic coverage** non sono presenti bug nel codice: il codice è scritto correttamente e funziona come previsto, tuttavia **il test non viene superato**. È una situazione in cui JUnit segnala un errore, ma in realtà l'errore non è nel codice. Anche in questo caso, la copertura logica deve essere sempre del 100%.

(Nell'ottimista invece non trovi il bug ma il bug c'è).

Se una classe espone un'operazione come pubblica, significa che chiunque può invocarla liberamente.

Di conseguenza, devo garantire che il suo utilizzo sia sempre corretto, indipendentemente da chi o cosa la richiami.

Supponiamo, ad esempio, di avere un metodo **divisione**.

Se questo metodo viene utilizzato da un **controllore**, e il controllore gestisce internamente il caso

di divisione per zero, allora **testando il metodo tramite il controllore non si raggiunge alcuna failure**, perché la divisione per zero viene evitata a monte. Tuttavia, se **testo direttamente il metodo `divisione`**, senza passare per il controllore, e tale metodo **non effettua alcun controllo** sulla divisione per zero, allora **riesco a generare una divisione per zero**, causando un errore, e quindi ho una failure.

Secondo il Professor Falessi, l'esempio appena descritto **non rappresenta una buona pratica** di programmazione. Infatti, non è compito del controllore gestire la divisione per zero, poiché è il concetto stesso di “divisione” che deve sapere come comportarsi in tale situazione.

ESEMPIO GIUSTO

Un esempio più corretto e significativo è quello dei viaggi aerei andata e ritorno.

In questo caso, è il controllore che deve assicurarsi che la data di ritorno sia successiva alla data di partenza.

Di conseguenza, quando il controllore passa al metodo la **differenza tra le due date** (ossia il numero di giorni di viaggio), il metodo può **aspettarsi sempre un valore positivo**.

Ad esempio, il metodo `calcolaTariffa()` si aspetta di ricevere un numero positivo di giorni, poiché il controllo di coerenza sulle date è stato **già eseguito dal controllore**.

Pertanto, `calcolaTariffa()` **non deve verificare** se il valore è minore di zero:

se il numero di giorni risulta negativo, ciò indica un **errore nel comportamento del controllore**, non del metodo.

In altre parole, è il **controllore** che utilizza `calcolaTariffa()`, e che deve **garantire la correttezza dei dati** che vi transitano.

Nel caso in cui la data di ritorno sia anteriore alla data di partenza, la failure non si verifica, poiché il controllore intercetta e gestisce l'anomalia prima di chiamare `calcolaTariffa()`. Tuttavia, se il test viene condotto direttamente sul metodo `calcolaTariffa()`, la failure si

manifesta, mentre testando il metodo passando per il controllore, la failure non si presenta, in quanto l'errore viene gestito a monte.

NEI TEST

Se sto testando il **controllore** e passo un **ritorno antecedente all'andata**, mi aspetto che il controllore **gestisca correttamente l'anomalia**, evitando di proseguire con l'esecuzione del metodo successivo. Successivamente, posso decidere di testare direttamente il metodo **calcolaTariffa()** per verificare come si comporta di fronte a valori negativi.

Tuttavia, in questo caso, **il test fallisce**, e ciò è **del tutto normale**: il metodo è infatti **progettato per non essere mai invocato con un numero di giorni negativo**.

In realtà il bug non c'è nel codice applicativo, ma nel test sembra che ci sia. Ecco perché annovero questo caso nel Pessimistic Coverage.

Sto testando un metodo con un valore che il controllore me lo rende **non possibile**.

Esempio Analogo

Immagina di avere un metodo chiamato **versaAcquaNelBicchiere(litri)**.

Questo metodo **si aspetta** che tu gli passi **un numero positivo**, perché ovviamente non puoi versare **-2 litri d'acqua**. Ora, se c'è un **controllore** che controlla prima quanta acqua vuoi versare, e **impedisce di passare un numero negativo**, allora tutto fila liscio. Se però tu decidi di testare **direttamente il metodo versaAcquaNelBicchiere(-2)**, lui, poverino, non sa cosa fare: **fallisce**.

Ma il punto è proprio questo: il metodo non è stato pensato per gestire valori negativi, **perché nella logica dell'applicazione non dovrebbero mai arrivarci**.

È come rimproverare un bicchiere perché non sa cosa fare se gli versi l'acqua "al contrario".